

Reporte de Práctica 2: Análisis y Desarrollo de una Aplicación Web con API

1. Portada

Institución: Universidad Politécnica de Aguascalientes

Materia: Tecnologías Y Aplicaciones En Internet

Práctica: Práctica 2. Análisis y Desarrollo de una Aplicación Web con API

Alumno: Jose Manuel Ortiz Medrano

Matrícula / ID: UP230598

Fecha de Entrega: 5 de junio de 2026

2. Objetivo

Diseñar y desarrollar una aplicación web funcional con arquitectura cliente-servidor, donde el frontend (React + TypeScript) se comunique con un backend propio construido como API REST (Django + Django REST Framework), todo orquestado mediante Docker. La idea fue poner en práctica el ciclo completo: modelar datos, exponer endpoints, consumirlos desde el frontend y presentar la información de manera útil al usuario final.

3. Investigación y Análisis

¿Qué es una API REST?

Una API REST (Representational State Transfer) es una interfaz que permite la comunicación entre sistemas a través del protocolo HTTP. Cada recurso se identifica con una URL y las operaciones se expresan mediante los métodos del protocolo: **GET** para leer, **POST** para crear, **PUT** para actualizar y **DELETE** para eliminar. La respuesta suele venir en formato JSON, que es fácilmente interpretable tanto por humanos como por máquinas.

Lo importante de REST es que es *stateless*: el servidor no recuerda nada del cliente entre peticiones, toda la información necesaria viene en cada solicitud. Esto lo hace más escalable y predecible.

¿Por qué Django REST Framework?

Django es uno de los frameworks de Python más usados para backend web. Viene con un ORM muy potente que abstrae las consultas SQL, un sistema de migraciones, administración automática y muchas utilidades listas para usar. Django REST Framework (DRF) extiende Django para facilitar la creación de APIs: serializa modelos, genera rutas automáticamente con **ViewSet** y **Routers**, y tiene manejo de permisos integrado.

Para este proyecto era una buena elección porque no queríamos reinventar la rueda: el **ModelViewSet** de DRF genera los cinco endpoints CRUD con muy poco código, y el **ModelSerializer** convierte el modelo de base de datos a JSON sin esfuerzo.

¿Por qué React con TypeScript y Vite?

React es la librería de interfaces más popular actualmente. Su modelo de componentes y el estado reactivo permiten construir UIs dinámicas sin recargar la página. TypeScript añade tipado estático sobre JavaScript, lo que hace el código más robusto y detecta errores en tiempo de compilación antes de que lleguen al navegador.

Vite reemplaza a webpack como bundler/servidor de desarrollo y es notablemente más rápido, tanto al arrancar como al actualizar el módulo en caliente (Hot Module Replacement). Para este proyecto, eso se notó en el tiempo de iteración al desarrollar.

¿Por qué Docker?

El problema clásico en desarrollo es "en mi máquina sí funciona". Docker resuelve eso encapsulando la aplicación y todas sus dependencias dentro de contenedores. Con un solo archivo `docker-compose.yml` levantamos los tres servicios (PostgreSQL, Django y React/Vite) con sus configuraciones exactas, sin importar el sistema operativo del desarrollador.

Para este proyecto fue especialmente útil porque coordinar la versión de Python, Node, PostgreSQL y sus librerías manualmente es propenso a errores. Docker eliminó ese problema desde el inicio.

Análisis del Dominio: Sistema de Gestión de Inventario

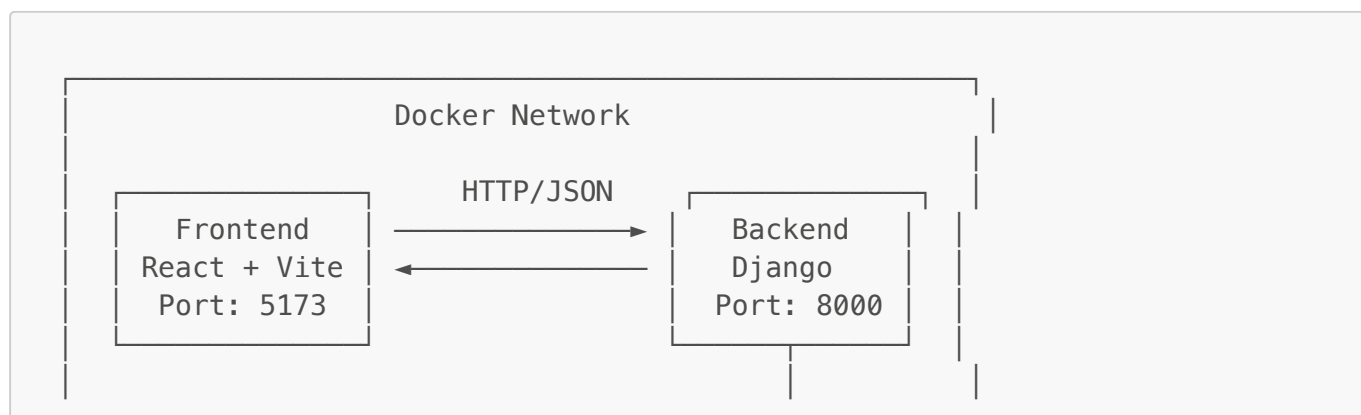
Se decidió construir **TechStore**, un sistema de gestión de inventario para productos tecnológicos. El dominio fue sencillo de modelar: un **Producto** tiene nombre, categoría, precio, stock, proveedor, fecha de registro y estado. Con esos campos se puede cubrir un caso de uso realista y completo.

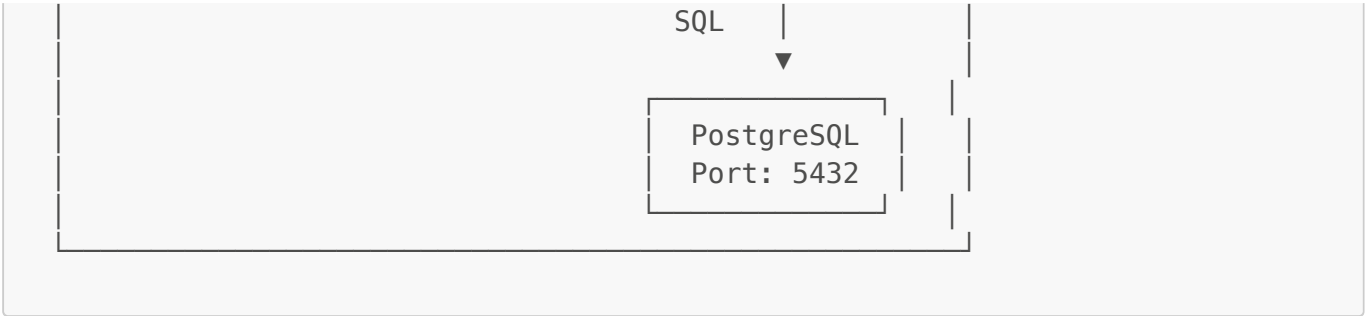
Las funcionalidades identificadas fueron:

- Autenticación con sesión persistente (JWT simulado en cliente).
- Dashboard con KPIs calculados del inventario.
- Listado de productos con búsqueda y filtrado por categoría.
- Alta de nuevos productos mediante formulario validado.
- Eliminación de productos.
- Exportación del inventario a Excel y PDF.

4. Diagramas

4.1 Arquitectura General del Sistema





4.2 Modelo de Datos (Backend)

Producto	
id	: AutoField (PK)
nombre	: CharField(200)
precio	: DecimalField(10,2)
categoria	: CharField(100)
stock	: IntegerField
proveedor	: CharField(200)
fechaRegistro	: DateField
estado	: CharField(50)

4.3 Endpoints de la API REST

Método	URL	Acción
GET	/api/productos/	Lista todos los productos
POST	/api/productos/	Crea un nuevo producto
GET	/api/productos/{id}/	Obtiene un producto por su ID
PUT	/api/productos/{id}/	Actualiza un producto por ID
DELETE	/api/productos/{id}/	Elimina un producto por ID

Todos estos endpoints fueron generados automáticamente por el **DefaultRouter** de Django REST Framework al registrar el **ProductoViewSet**.

4.4 Flujo de Comunicación Frontend ↔ Backend



```

▼
App.tsx → fetch("http://localhost:8000/api/productos/")
├── GET /api/productos/ → Django ViewSet → PostgreSQL
│   └── ← JSON con lista de productos ─┘
├── POST /api/productos/ → Serializer → Model.save() → DB
└── DELETE /api/productos/{id}/ → QuerySet.delete() → DB

```

4.5 Estructura de Carpetas del Proyecto

```

Practica_2UPA/
├── docker-compose.yml          ← Orquestador principal
├── GUIA_DOCKER.md
├── Backend/
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── manage.py
│   ├── config/                ← Configuración Django
│   └── productos/             ← App principal
│       ├── models.py          ← Modelo Producto
│       ├── serializers.py      ← Conversión a JSON
│       ├── views.py            ← ProductoViewSet
│       └── urls.py             ← Registro de rutas
├── Fronted/
│   └── techstore-frontend/
│       ├── Dockerfile
│       ├── package.json
│       └── src/
│           ├── App.tsx         ← Lógica principal + Inventario
│           ├── context/
│           │   └── AuthContext.tsx
│           ├── components/
│           │   ├── Login.tsx
│           │   ├── Dashboard.tsx
│           │   └── FormProducto.tsx
│           └── types/
│               └── producto.ts

```

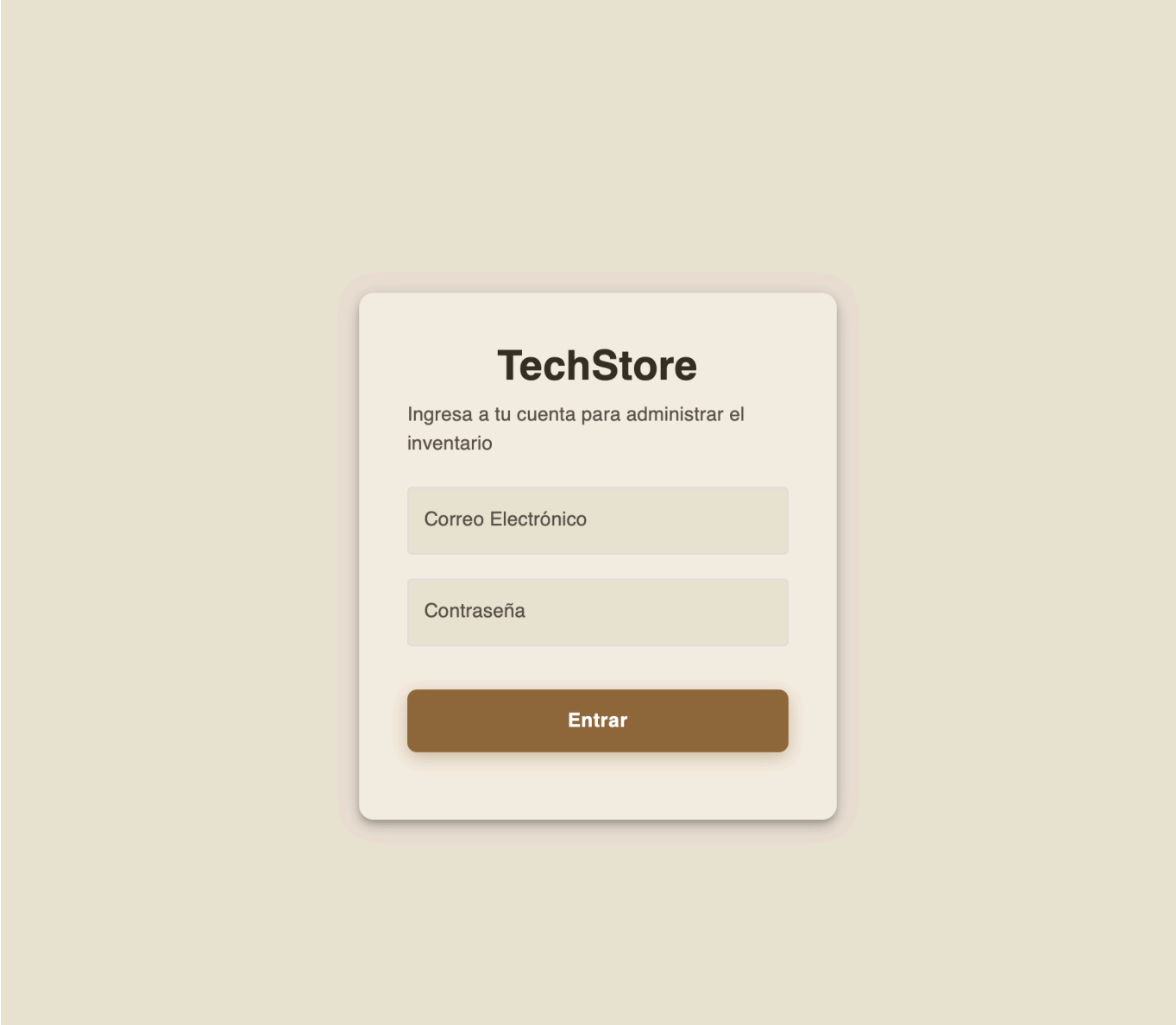
5. Capturas de Funcionamiento

Nota: Las siguientes capturas muestran la aplicación corriendo localmente con Docker Compose en <http://localhost:5173>.

5.1 Pantalla de Login

La pantalla de inicio de sesión tiene un diseño con efecto glassmorphism (fondo con blur). La validación del formulario está hecha con React Hook Form + Zod, así que si el usuario intenta enviar el formulario

vacío, los errores aparecen en tiempo real sin recargar la página.





The image shows a login form for 'TechStore'. The form is centered on a light beige background. It has a title 'TechStore' in a large, bold, black font. Below the title is a subtitle 'Ingresa a tu cuenta para administrar el inventario' in a smaller, black font. There are two input fields: the first is labeled 'Correo Electrónico' and contains a single vertical bar as a placeholder; the second is labeled 'Contraseña'. Both fields have red error messages below them: 'El correo electrónico es obligatorio' and 'La contraseña debe tener al menos 6 caracteres'. A brown 'Entrar' button is at the bottom of the form.

TechStore

Ingresa a tu cuenta para administrar el inventario

Correo Electrónico

El correo electrónico es obligatorio

Contraseña

La contraseña debe tener al menos 6 caracteres

Entrar

5.2 Dashboard — Panel de Control

Al autenticarse, la vista principal muestra tres KPIs calculados directamente desde los datos de la API:

- **Total de Productos** registrados.
- **Valor total del Inventario** (suma de precio $\times$ stock de cada producto).
- **Alertas de Stock Bajo** (productos con menos de 5 unidades).

Debajo de los KPIs hay una gráfica de barras generada con Recharts que muestra la distribución de productos por categoría.



5.3 Vista de Inventario con Filtros

La sección de inventario muestra la tabla completa de productos obtenidos del endpoint **GET /api/productos/**. Desde aquí se puede:

- Buscar por nombre o proveedor (filtrado en tiempo real en el cliente).
- Filtrar por categoría usando el selector desplegable.
- Exportar el listado actual a **Excel** (usando la librería **xlsx**) o **PDF** (usando **jsPDF** con **jspdf-autotable**).
- Eliminar un producto directamente desde la fila.

Lista de Productos

Buscar producto o proveedor...

Categoría
Todas

ACTUALIZAR DATOS

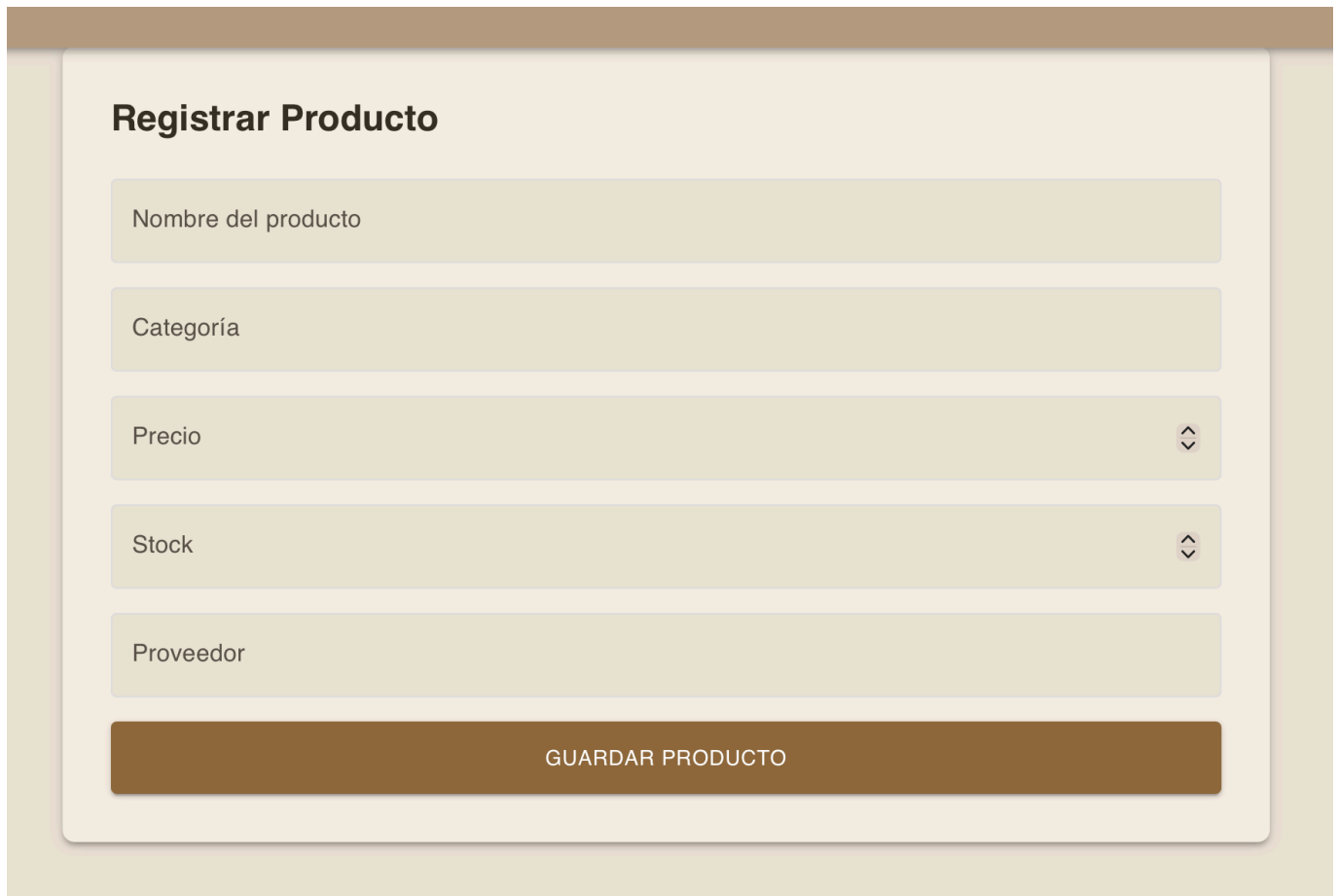
EXCEL

PDF

ID	Nombre	Categoría	Precio	Stock	Proveedor	Fecha Reg.	Estado	Acción
1	Laptop HP 15	Electrónica	\$14500.00	8	HP México	2026-06-05	Activo	ELIMINAR
2	Laptop Lenovo IdeaPad	Electrónica	\$13200.00	6	Lenovo México	2026-06-05	Activo	ELIMINAR
3	Mouse Logitech M170	Accesorios	\$250.00	35	Logitech	2026-06-05	Activo	ELIMINAR
4	Teclado Mecánico Redragon	Accesorios	\$850.00	20	Redragon	2026-06-05	Activo	ELIMINAR
5	Monitor Samsung 24 pulgadas	Electrónica	\$3200.00	10	Samsung	2026-06-05	Activo	ELIMINAR
6	Memoria USB Kingston 64GB	Almacenamiento	\$180.00	50	Kingston	2026-06-05	Activo	ELIMINAR
7	Disco Duro Externo 1TB	Almacenamiento	\$1250.00	15	Seagate	2026-06-05	Activo	ELIMINAR
8	SSD Kingston 480GB	Almacenamiento	\$750.00	18	Kingston	2026-06-05	Activo	ELIMINAR

5.4 Formulario de Alta de Producto

El formulario de registro de nuevo producto incluye validación de campos requeridos. Al guardar exitosamente, hace una petición **POST** al backend y redirige automáticamente a la vista de inventario actualizada.



El formulario 'Registrar Producto' está diseñado con un fondo beige claro y un encabezado de color marrón. Contiene cinco campos de entrada de texto: 'Nombre del producto', 'Categoría', 'Precio', 'Stock' y 'Proveedor'. Los campos 'Precio' y 'Stock' tienen iconos de flechas hacia arriba y abajo a la derecha, lo que sugiere que son campos de tipo número con validación de rango. Debajo de los campos, hay un botón rectangular de color marrón con el texto 'GUARDAR PRODUCTO' en mayúsculas blancas.

Registrar Producto

Nombre del producto

Categoría

Precio

Stock

Proveedor

GUARDAR PRODUCTO

5.5 API REST en el Navegador

Django REST Framework incluye un explorador web de la API accesible en

<http://localhost:8000/api/productos/>, donde se pueden ver los datos en JSON y probar los endpoints directamente.

```
GET /api/productos/
```

```
HTTP 200 OK
```

```
Allow: GET, POST, HEAD, OPTIONS
```

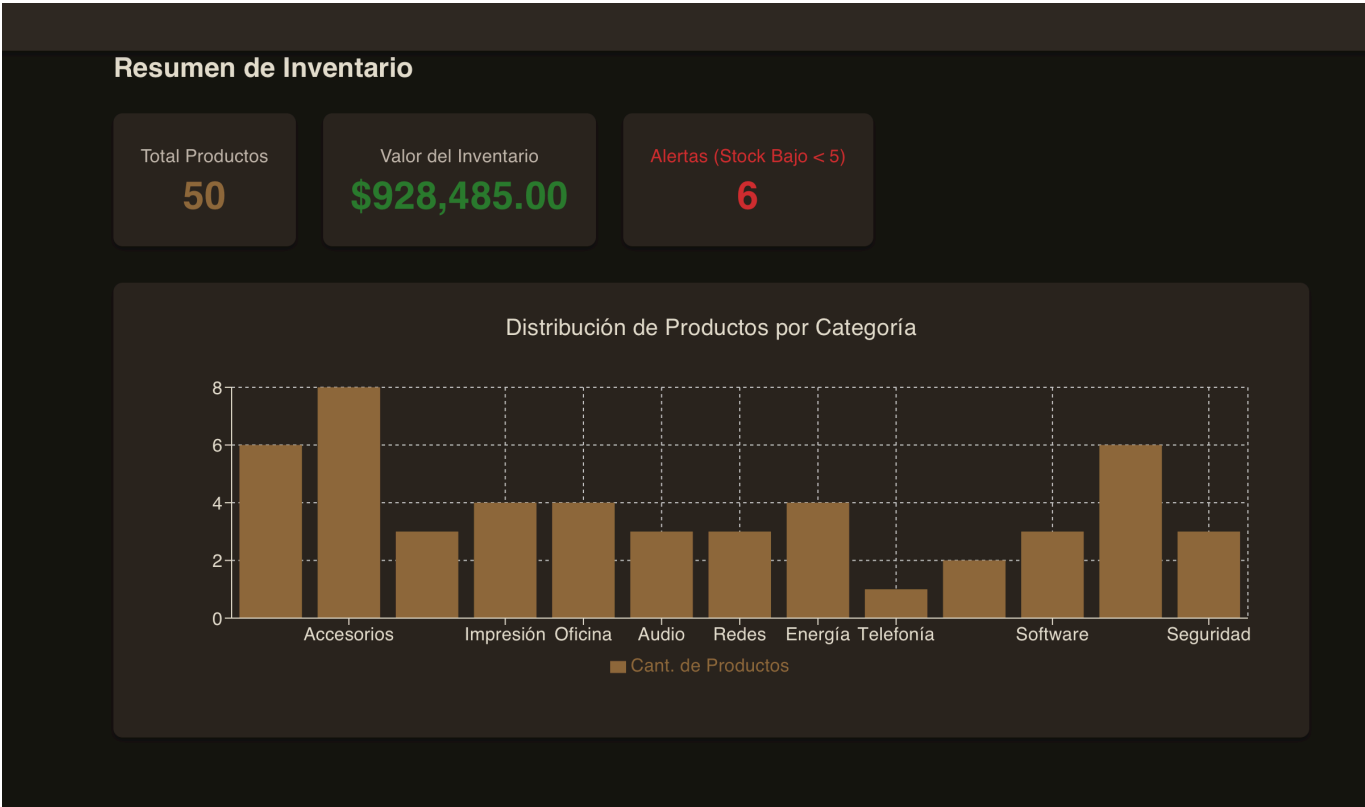
```
Content-Type: application/json
```

```
Vary: Accept
```

```
[
  {
    "id": 1,
    "nombre": "Laptop HP 15",
    "precio": "14500.00",
    "categoria": "Electrónica",
    "stock": 8,
    "proveedor": "HP México",
    "fechaRegistro": "2026-06-05",
    "estado": "Activo"
  },
  {
    "id": 2,
    "nombre": "Laptop Lenovo IdeaPad",
    "precio": "13200.00",
    "categoria": "Electrónica",
    "stock": 6,
    "proveedor": "Lenovo México",
    "fechaRegistro": "2026-06-05",
    "estado": "Activo"
  },
  {
    "id": 3,
    "nombre": "Mouse Logitech M170",
    "precio": "250.00",
    "categoria": "Accesorios",
    "stock": 35,
    "proveedor": "Logitech",
    "fechaRegistro": "2026-06-05",
    "estado": "Activo"
  },
  {
    "id": 4,
    "nombre": "Teclado Mecánico Redragon",
    "precio": "850.00",
    "categoria": "Accesorios",
  }
```

5.6 Modo Oscuro

La aplicación tiene soporte completo de modo oscuro, persistido en **localStorage**. El cambio se puede hacer desde el botón en el AppBar y afecta todos los componentes incluyendo el sidebar, las tarjetas de KPIs y la tabla.



Gestión de Inventario

admin@techstore.com

Lista de Productos

Buscar producto o proveedor...

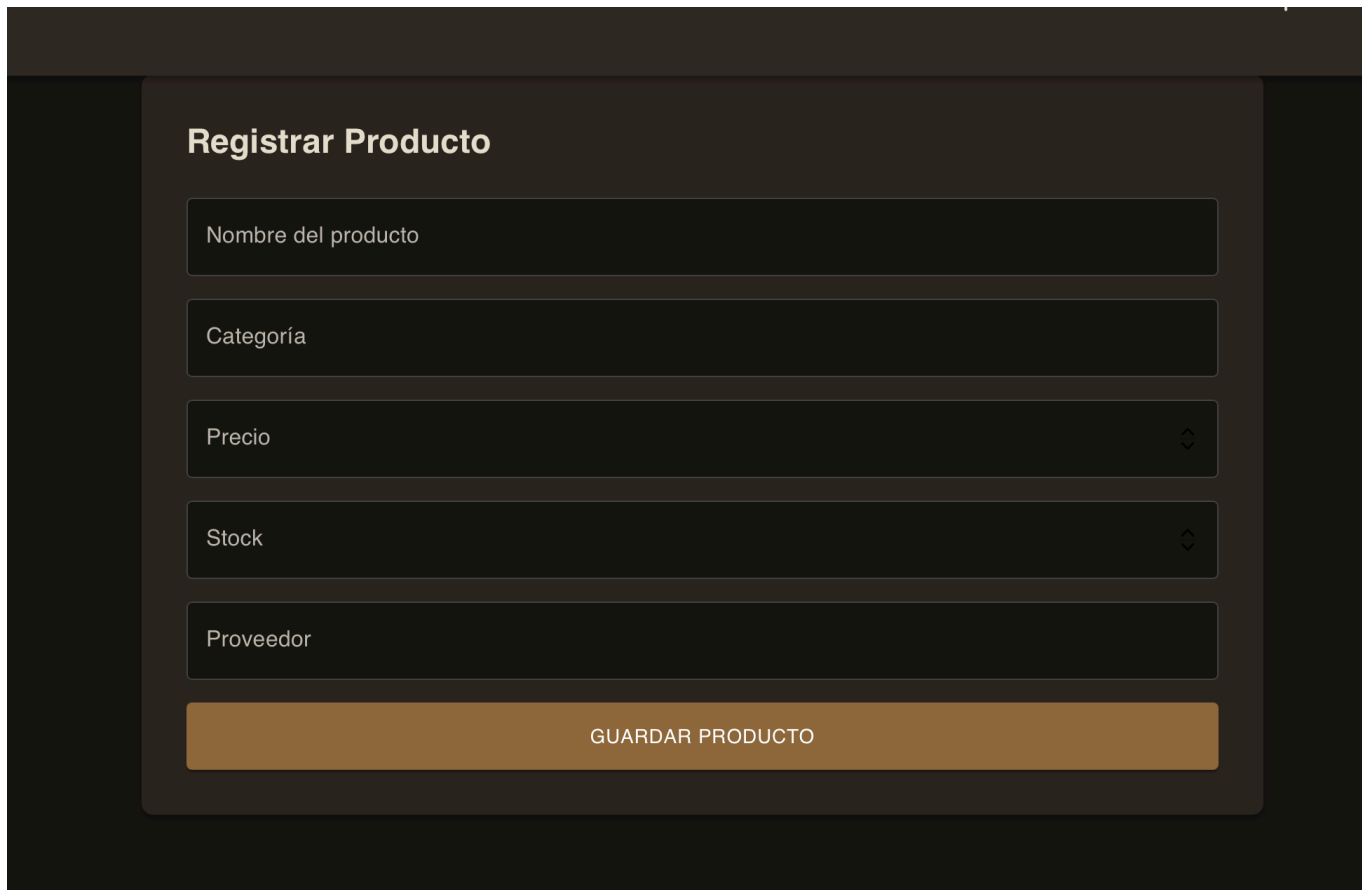
Categoría Todas

ACTUALIZAR DATOS

EXCEL

PDF

ID	Nombre	Categoría	Precio	Stock	Proveedor	Fecha Reg.	Estado	Acción
1	Laptop HP 15	Electrónica	\$14500.00	8	HP México	2026-06-05	Activo	ELIMINAR
2	Laptop Lenovo IdeaPad	Electrónica	\$13200.00	6	Lenovo México	2026-06-05	Activo	ELIMINAR
3	Mouse Logitech M170	Accesorios	\$250.00	35	Logitech	2026-06-05	Activo	ELIMINAR
4	Teclado Mecánico Redragon	Accesorios	\$850.00	20	Redragon	2026-06-05	Activo	ELIMINAR
5	Monitor Samsung 24 pulgadas	Electrónica	\$3200.00	10	Samsung	2026-06-05	Activo	ELIMINAR
6	Memoria USB Kingston 64GB	Almacenamiento	\$180.00	50	Kingston	2026-06-05	Activo	ELIMINAR
7	Disco Duro Externo 1TB	Almacenamiento	\$1250.00	15	Seagate	2026-06-05	Activo	ELIMINAR



The image shows a web form titled "Registrar Producto" (Register Product). It contains five input fields stacked vertically: "Nombre del producto" (Product Name), "Categoría" (Category), "Precio" (Price), "Stock", and "Proveedor" (Supplier). The "Precio" and "Stock" fields have small downward arrows on their right sides, indicating they might be dropdown menus or have a specific format. Below the input fields is a large, orange button labeled "GUARDAR PRODUCTO" (Save Product).

6. Aprendizajes y Dificultades

Aprendizajes

Sobre la arquitectura desacoplada:

Lo que más me quedó claro fue cómo separar responsabilidades entre el frontend y el backend. El backend no sabe nada de cómo se ve la interfaz, solo expone datos en JSON. El frontend no sabe nada de la base de datos, solo consume la API. Esa separación hace que puedas cambiar uno sin romper el otro, algo que en teoría suena obvio pero que en la práctica se siente diferente cuando lo vives construyendo.

Sobre Django REST Framework:

Descubrí que un `ModelViewSet` con su `DefaultRouter` genera los cinco endpoints CRUD con literalmente cinco o seis líneas de código. El `ModelSerializer` con `fields = '__all__'` también ahorra mucho tiempo. Claro que en proyectos reales hay que ser más selectivo con los campos expuestos, pero para este ejercicio fue perfecto ver cuánto hace DRF por ti.

Sobre el CORS:

Cuando el frontend (localhost:5173) intentó hablar con el backend (localhost:8000), el navegador bloqueó las peticiones por política de CORS. Hubo que instalar `django-cors-headers` y configurar los orígenes permitidos en el settings de Django. Es un concepto que ya conocía de nombre pero nunca lo había resuelto yo mismo.

Sobre Docker Compose:

Entender el `docker-compose.yml` fue bueno. Ver cómo los servicios se referencian entre ellos por nombre (`db`, `backend`, `frontend`), cómo los volúmenes persisten los datos de PostgreSQL y cómo la

opción `depends_on` controla el orden de arranque. Ahora tiene mucho más sentido cuando veo un `docker-compose.yml` en un repositorio de verdad.

Sobre TypeScript en React:

Tipar los props de los componentes y los datos que vienen de la API fue un poco más de trabajo al inicio, pero cuando en algún momento cambié algo en el tipo `Producto`, TypeScript me marcó de inmediato todos los lugares donde usaba ese tipo incorrectamente.

Dificultades

Configurar CORS correctamente:

Fue la primera cosa que rompió. El frontend y el backend corren en puertos distintos, así que sin la configuración de CORS, el navegador rechaza las peticiones por seguridad. Tuve que entender bien por qué sucede (la misma política de origen del navegador) y no solo copiar la solución, porque hay que saber qué `ALLOWED_ORIGINS` poner según el entorno.

La conexión entre Django y PostgreSQL dentro de Docker:

En un inicio, Django intentaba conectarse a `localhost` para encontrar la base de datos, pero dentro de Docker el host de Postgres es el nombre del servicio (`db`), no `localhost`. Ese detalle pequeño hizo que el backend no arrancara hasta que entendí cómo funciona la red interna de Docker.

Sincronizar los campos del modelo con el frontend:

El modelo `Producto` fue evolucionando durante el desarrollo. Primero no tenía `stock` ni `proveedor`, y el frontend esperaba esos campos. Cada vez que se agrega un campo al modelo hay que crear y aplicar una migración en Django, lo que dentro de Docker requiere entrar al contenedor o lanzar el comando desde afuera. Eso fue algo que al principio me tomó tiempo pero luego se volvió rutinario.

Estado global de autenticación:

Implementar un `AuthContext` con React Context API para que el login persista entre recargas (usando `localStorage`) fue más complejo de lo esperado. El reto fue sincronizar el estado de `isLoading` para no mostrar la pantalla de login por un instante cuando la página carga y está revisando si hay sesión guardada.

7. Conclusiones

Esta práctica fue la primera vez que armo algo donde el frontend y el backend son proyectos completamente separados que se tienen que comunicar. Antes siempre había trabajado con todo junto, así que ver cómo una petición sale del navegador, llega a Django, consulta la base de datos y regresa como JSON fue algo que en papel ya sabía pero que se entiende diferente cuando lo ves funcionando de verdad.

Lo que no esperaba fue lo mucho que influye la configuración en que todo funcione. El CORS, el host de la base de datos dentro de Docker, el orden en que arrancan los servicios... nada de eso es lógica de negocio, pero si está mal, nada corre. Esa parte me tomó más tiempo del que pensaba.

En general fue una práctica que valió la pena, sobre todo porque el resultado es algo que puedes levantar en cualquier computadora con un solo comando y funciona igual.

8. Bibliografía

- Django Software Foundation. (2024). Django documentation. <https://docs.djangoproject.com/>
- Encode. (2024). Django REST Framework documentation. <https://www.django-rest-framework.org/>
- Meta Open Source. (2024). React documentation. <https://react.dev/>
- Material UI. (2024). MUI documentation. <https://mui.com/material-ui/>
- Vite. (2024). Vite documentation. <https://vite.dev/>
- Docker Inc. (2024). Docker Compose documentation. <https://docs.docker.com/compose/>
- XLSX.js Contributors. (2024). SheetJS Community Edition. <https://sheetjs.com/>
- jsPDF Contributors. (2024). jsPDF documentation. <https://rawgit.com/MrRio/jsPDF/master/docs/>